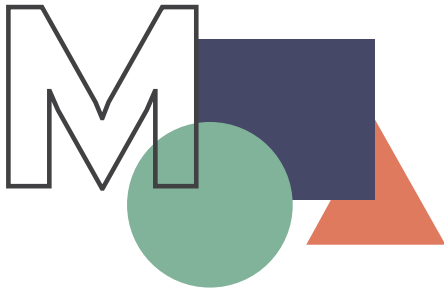


Manim: Creating Dynamic Visual Animations

Learn about how Manim, a Python library, converts code into captivating animations, and why it has become the preferred choice for educators, developers, and content creators worldwide.



Modern educational videos in mathematics and science often feature stunning animations that turn complex ideas into visual masterpieces. Equations flow dynamically, geometric figures evolve seamlessly, and algorithms come alive through elegant motion—making learning both engaging and intuitive.

Traditionally, educators depended on costly proprietary tools like Adobe After Effects or specialised mathematical visualisers with steep learning curves and high licensing fees. These barriers often left creative educators confined to static presentations and textbook-based explanations.

Today, the open source revolution has transformed this space with Manim—a Python library that makes high-quality educational animations accessible to all. Originally built by Grant Sanderson for his *3Blue1Brown* channel, Manim has grown into a powerful engine enabling anyone with basic programming skills to create professional grade visualisations.

Understanding Manim's architecture

Before creating animations, it's important to understand Manim's core components.

- **Scenes:** The canvas and timing controller for animations.
- **Mobjects:** Mathematical objects representing visual elements.
- **Animations:** Transformations defining how objects change over time.
- **Rendering pipeline:** Converts code into high-quality video files.

Scene structure: Manim operates on a scene-based architecture, where every animation is defined as a Python class inheriting from Scene.

Its key characteristics are:

- Must contain a `construct()` method.
 - Acts as both a spatial canvas and a timing controller.
 - Provides specialised scene types for different contexts.
- Here's an example scene structure:

```
class BasicDemo(Scene):
    def construct(self):
        # Scene initialization
        title = Text("Manim Architecture")

        # Animation sequence
        self.play(Write(title))
        self.wait(2)
```

You can refer to Figure 1 for the output.

Mobjects (Mathematical objects): Mobjects represent all the visual elements in Manim animations.

Basic primitives are:

- Geometric shapes → `Circle()`, `Square()`, `Arrow()`, `Polygon()`
- Text rendering → `Text()`, `Tex()` (with LaTeX support)
- Graphs and plots → `Axes()`, `FunctionGraph()`, `NumberPlane()`

Advanced constructs are:

- 3D objects → `ThreeDAxes()`, `Surface()`, `ParametricSurface()`
- Custom graphics → `SVGObject()`, `ImageObject()`

Grouping mechanisms are:

- Vertical grouping for synchronised animations → `VGroup()`
- General grouping for treating multiple objects as one → `Group()`



Figure 1: Manim architecture